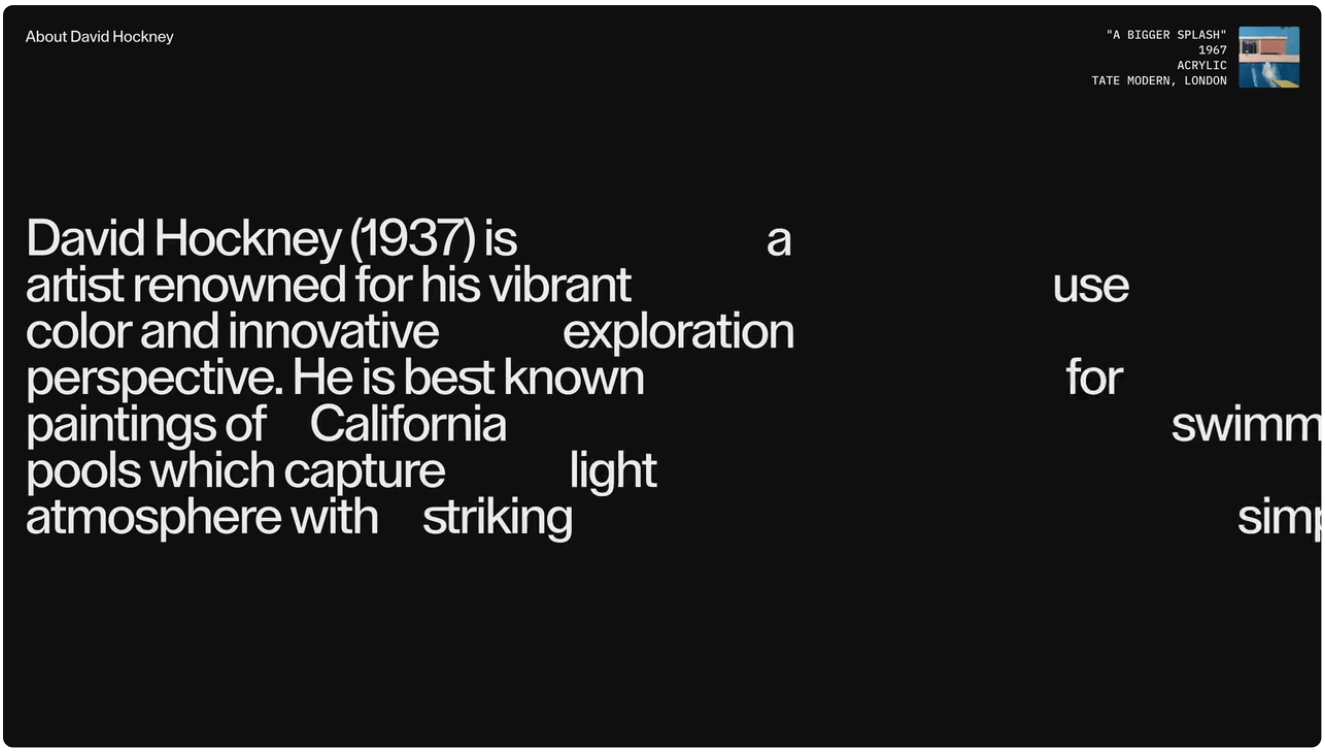
Made With Gsap

004
Simultaneous words



For this effect, I wanted to make the words of a paragraph appear in a new way. Instead of having them appear from top to bottom, I made them slide from right to left with a stacking effect. The words will slide in from outside the screen to their final position, all linked to the user's scroll.

HTML Structure

To make the effect easy to implement, you just need to write text inside an element without worrying about separating each word into a span. This element will be the child of a div that we'll keep pinned to the user's scroll.

```
<div class="pin">
    <div class="container">
        <p class="paragraph">David Hockney (1937) is a British artist.</p>
    </div>
</div>
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS
Made With Gsap

ective lines once separated, I
ided.

```
document.fonts.ready.then(() => {
    // Once the font is loaded, let the fun begin!
    init()
})
```

Once the font is loaded, let the fun begin! I created a utility function that will wrap each word in a span. This function takes the DOM element containing the sentence to separate.

```
function wrapWordsInSpan(element) {
    const text = element.textContent
    element.innerHTML = text
        .split(' ')
        .map(word => `<span class="word">${word}</span>`)
        .join(' ');
}
```

Pinning the container

To begin, we can set the height of the container section in CSS. The effect will take place over this entire height. The shorter this height is, the faster the words will appear as the user scrolls.

```
.mwg_effect004 .pin-height {
    height: 500vh;
}
```

I then pin the container division to the full height of pin-height using a `ScrollTrigger.create()` instance. This instance allows us to define when the division should be pinned.

```
const pinHeight = document.querySelector('.mwg_effect004 .pin-height')
const container = document.querySelector('.mwg_effect004 .container')
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

```
    end: "bottom bottom",
    pin: container
})
```

To explain the code above: when the pin-height division reaches the top of the viewport, we start pinning the container until pin-height moves past the bottom of the viewport. This way, the container remains perfectly visible on the screen throughout this distance.

For a different approach, we could avoid pinning this div and let the sentence appear while staying within the natural flow of the page's scroll. The only downside to skipping this step is that if the sentence is long, the animation will be too fast.

Storing each line

The idea is that the animation progresses simultaneously for each line. To do this, I first stored each word within the line it corresponds to.

```
// we will have an array of lines that contain an array of words
const lines = [[]]
let lineIndex = 0

for (let i = 0; i < words.length; i++) {
    const word = words[i]
    // distance of the top outer border of the word to to the top edge of its parent
    const offsetTop = word.offsetTop

    // if distance is different from previous word we start a new line
    if (i > 0 && offsetTop !== words[i - 1].offsetTop) {
        lines.push([])
        lineIndex++
    }
    lines[lineIndex].push(word)
}
```

If a word is further from the top of the screen than the previous word, it's on a new line. The code above follows this logic and creates an array of lines, where each line contains its respective words.

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

Step 1: Moving Words Off-Screen. To start, we'll move the words out of view. Each word is shifted 100% of the viewport width to the right. Additionally, we ensure the words are `display: inline-block`, as transforms won't work otherwise.

```
.mwg_effect004 .word {
    display: inline-block;
    transform: translate(100vw, 0);
}
```

Step 2: Animating the Words. Now, we'll evenly distribute the appearance of each word across the scroll distance. To avoid all words animating simultaneously, I've added `stagger: 0.2`.

```
lines.forEach(lineWords => {
    gsap.to(lineWords, {
        x: 0,
        stagger: 0.2,
        ease: 'power1.inOut',
        scrollTrigger: {
            trigger: '.mwg_effect004',
            start: 'top top',
            end: 'bottom bottom',
            scrub: true
        }
    })
});
```

But why 0.2? The key here is understanding that the default duration for a tween is `0.5`. Since I didn't specify a duration, the default one is used. If each animation takes 0.5 seconds, setting `stagger: 0.5` would make each word start as soon as the previous one finishes. With `scrub: true` in a ScrollTriggered animation, the animation's duration adjusts to fit perfectly between the start and end scroll positions.

So, if we set stagger to 0.2, one word will appear just as the previous one finishes its movement. Reducing this value allows multiple words per line to overlap at the same time.

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

. `ease` values to tweak the motion. For example, using `expo.out` makes the words arrive quickly at their final positions and then slow down gently.

In this example, I used the Lenis smooth scroll to further enhance the fluidity and sliding effect. If you'd like to implement it as well, visit our Go Further page to learn how to easily add it.

Final code

HTML CSS JAVASCRIPT

```html
<section class="mwg_effect004">
    <p class="scroll">Scroll</p>

    <div class="pin-height">
        <div class="container">
            <div class="header">
                <p class="left">About David Hockney</p>
                <div class="right">
                    <p>"A bigger splash" <br/>1967 </br>acrylic <br />Tate Modern, Lond
                    <img src="assets/medias/painting.jpg" alt="">
                </div>
            </div>
            <p class="paragraph">David Hockney (1937) is a British artist renowned for
        </div>
    </div>
</section>
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

omething else?

BACK TO THE COLLECTION

Never miss what's next

Your email

SUBMIT

By submitting your email, you'll be the first to know about upcoming updates for Made With Gsap. You can unsubscribe at any time.